Shorter proofs for K215 and CM215

Checked transport, exact data compaction,
and the search for 214 states

Research packet for Vladimir Reshetnikov's *ProveIt*
21 October 2026

Abstract

The most useful next simplification is to prove nearby machines correct by a local halting-equivalence theorem, reusing the calibrated K215 endpoint. For the existing conditional-marker alternative CM215, this packet supplies a history-free invariant, a reusable positive-stuttering theorem checked in both Lean and Coq, and an exact-table Lean proof of all-tape equihalting from the designated start control. The source invariant has only three clauses; target blocks have length one or three. Separately, exact-data pilots reduce the current Lean source-data module by 47.49% and the Rocq offset-certificate module by 84.66%, with kernel-checked equality audits. These are source-size reductions, not measured checking-time gains.

No 214-state machine was constructed. Completed finite-window and source-abstraction experiments are supplied with explicit bounds and negative results. A concrete next target is to reorder the CM215 selector tests, but it remains a multi-row allocation problem. The new native-machine theorem is standalone: its representation adapter to the existing ProveIt endpoint, the final composed ZFC endpoint, and full upstream rebuilds of the data pilots remain integration work.

1 Results, evidence, and scope

The principal repository snapshot for this packet is `b6c7087477e51c017003bd729e712c8969733223`. Some retained research notes record the earlier retrieval pin `65e00c64...`; the analyzed machine inputs were subsequently hash-matched to the principal snapshot. This matters because the current development already includes substantial proof simplifications. The maintained Lean endpoint is a short application of shared infrastructure, not a long monolithic proof awaiting a cosmetic rewrite [2, 4]. The appropriate targets are repeated data, duplicated semantic arguments across candidates, and candidate-specific physical verification.

Result	Evidence delivered	Boundary
Positive-stuttering transport	Lean 4.32.0: no axioms. Coq 8.18.0: closed context, independent `coqchk` pass.	Generic transition theorem, independent of ZFC.
Exact K215–CM215 equivalence	All 24 Lean modules pass; literal tables, 215-state counts, six changed rows, all-tape theorem.	Standalone native representation; endpoint adapter pending.
Lean shared-tree pilot	456,548 to 239,744 bytes; two exact `rfl` audits, no axioms.	Changed upstream module and its consumers not rebuilt here.
Rocq path-compaction pilot	1,499,170 to 229,933 bytes; whole-certificate reflexivity proof and `coqchk`.	Checked with Coq 8.18.0; upstream uses Rocq 9.1.0.
214-state search	Complete receipts for specified local search and abstract source analysis.	No successful 214-state table; no lower bound.
Resolved source-CFG refactor	Precise proposed semantic interface and adoption sequence.	Proposal, not an implemented theorem.

1.1 What the consistency statement means here

Write $\mathrm{Ref}_{\mathrm{ZFC}}$ for the existence of a syntactic ZFC derivation of contradiction, using the repository's encoding. The existing endpoint states

$$\mathrm{Halts}(K215, \mathrm{blank}) \iff \mathrm{Ref}_{\mathrm{ZFC}}\,.$$

Negating both sides gives the desired nonhalting/consistency equivalence. The new theorem establishes

$$\mathrm{Halts}(K215, 0, h, \tau) \iff \mathrm{Halts}(CM215, 0, h, \tau) \tag{1}$$

for every integer head position h and every whole binary tape τ. Once the tape/run conventions and state labels are connected to the repository's formal machine, equation (1) composes directly with the existing endpoint.

The local proof uses only finite transition tables and elementary induction, and can be formalized in ZFC. The supplied machine-checked result is nevertheless an external Lean theorem, not a separately generated derivation encoded in ZFC syntax. This matches the repository's distinction between external proof-assistant endpoints and internal formal derivations [3, 4]. No consistency premise is needed to prove the equivalence, and no unconditional nonhalting assertion is added.

1.2 Attribution and novelty

CM215, its six-row transformation, the original marker argument, the candidate source, and the underlying compiler/calculus already belong to the repository [5]. They are not presented as newly discovered here. This packet contributes an independently written three-clause invariant, reusable transport proofs, a standalone exact-table Lean instantiation, two checked data-compaction pilots, and the recorded bounded experiments. The finite-window search method and the identified dead source branches also have repository antecedents [16]. Upstream license and attribution files are preserved with the inputs.

2 Simplify proofs by transporting an established endpoint

A nearby physical machine need not repeat source semantics, arithmetic calibration, source-to-runtime correspondence, and blank-start calculations. Instead, prove that its halting behavior agrees with the already-proved machine. The useful interface allows a noninjective configuration map and a positive finite number of target steps per source step.

2.1 A small, reusable theorem

Let $f : A \to \mathrm{Option}(A)$ and $g : B \to \mathrm{Option}(B)$ be deterministic partial transition functions. Write $a \xrightarrow[f]{n} a'$ for exactly n successful f-steps. A configuration halts if some finite successful run reaches a configuration whose next step is undefined. Let $F : A \to B$ be any map and $I \subseteq A$ a source invariant.

Theorem 1 (Positive-stuttering transport)**.** *Suppose that:*

1. *if* $I(a)$ *and* $f(a) = \text{some}(a')$*, then* $I(a')$*;*
2. *if* $I(a)$ *and* $f(a) = \text{none}$*, then* $g(F(a)) = \text{none}$*;*
3. *if* $I(a)$ *and* $f(a) = \text{some}(a')$*, then there is an integer* $m > 0$ *with* $F(a) \xrightarrow[g]{m} F(a')$*.*

Then, for every a *with* $I(a)$*,*

$$\text{Halts}_f(a) \iff \text{Halts}_g(F(a)).$$

Neither injectivity of F*, a target invariant, a reverse simulation, nor a uniform bound on* m *is required.*

Proof. For the forward direction, concatenate the finite blocks supplied by condition 3 along the source halting run. Condition 1 maintains the invariant; condition 2 makes the image of its final configuration terminal.

For the reverse direction, induct strongly on the length N of a halting target run starting at $F(a)$. If $f(a)$ is undefined, the source already halts. Otherwise $f(a) = \text{some}(a')$. Obtain a positive block of length m from condition 3. Determinism makes this block agree with the prefix of the halting target run. It cannot extend beyond its terminal configuration, because each of its transitions is successful. Thus $m \le N$, and cancellation leaves a halting suffix of length $N - m$ starting at $F(a')$. Since $m > 0$, $N - m < N$; apply the induction hypothesis using $I(a')$, then prepend the successful source step. □

Positivity is essential. With zero-length blocks, a nonhalting source self-loop could map to an already terminal target. The theorem's interface therefore makes strict positivity explicit rather than relying on an informal progress argument. The target's determinism is what makes a single forward simulation reflect halting; an unrelated nondeterministic branch could otherwise halt early.

2.2 Checked implementation

The Lean implementation is `proof_simplification/PositiveStuttering.lean`; it imports only `Init`. Its principal endpoint is

```
PositiveStuttering.Simulation.halts_iff
```

and its axiom report is empty. The corresponding Coq development is `proof_simplification/PositiveStuttering.v`. Coq 8.18.0 compiled it, printed "Closed under the global context", and `coqchk` rechecked it independently. Exact source hashes and logs are included.

This is the recommended interface for future local table optimizations. It separates two jobs: discovering a finite transformation certificate, which may use arbitrary search code, and checking its mathematical meaning, which belongs in the proof assistant. A successful search receipt alone does not establish the theorem. Each concrete instance must supply the invariant, successful blocks, and terminal preservation.

3 The easier additional 215-state candidate: CM215

CM215 is the best first additional endpoint *relative to the established K215 proof*. Its source program and logical seed are unchanged. Its incremental correctness proof can be a local bridge, importing the parent's source and consistency calibration. This does not claim that CM215 has the shortest complete proof from first principles: the transitive proof still includes K215, and the marker mechanism adds a local invariant.

3.1 Exact transformation

Use the native labels in the supplied machine files. A row $q/b \mapsto wDz$ means: in control q scanning b, write w, move in direction D, and enter z. There are exactly six changed rows:

Control / read	K215	CM215
52/0	$0\,R\,154$	$0\,L\,176$
156/1	$1\,R\,176$	$1\,R\,52$
165/0	$0\,R\,176$	$0\,R\,52$
176/0	$0\,R\,168$	$0\,R\,185$
176/1	$1\,R\,150$	$1\,R\,129$
193/0	$0\,R\,176$	$0\,R\,52$

Define $\phi(176) = 52$ and $\phi(q) = q$ otherwise, and

$$F(q, h, \tau) = (\phi(q), h, \tau).$$

This deliberately merges two logical controls. The physical control 176 is reused as a selector. Consequently the construction still has 215 working states: the logical merge does not free a physical state.

3.2 Replace predecessor history by a three-clause invariant

For an old configuration $c = (q, h, \tau)$, set

$$\begin{aligned} I(c) \quad \Longleftrightarrow \quad & (q = 52 \Rightarrow \tau(h-1) = 1) \\ & \wedge\, (q = 176 \Rightarrow \tau(h) = 1 \ \vee \ \tau(h-1) = 0) \\ & \wedge\, (q = 156 \Rightarrow \tau(h+1) = 1). \end{aligned}$$

No condition is imposed at other controls. In particular $I(0, h, \tau)$ holds for every tape and head position.

The third clause is the useful strengthening. A proof that remembers how control 176 was reached can instead retain the needed one bit at control 156. The exact table satisfies the following incoming-edge facts:

- Every edge into 52 writes one and moves right. The newly written one is therefore the destination's left marker.
- Every edge into 156 writes one and moves left. The newly written one is therefore immediately right of the destination head. In the concrete table its only incoming edge is $189/1 \mapsto 1L156$.
- Every edge into 176 either writes zero and moves right, or is the exceptional $156/1 \mapsto 1R176$. The first case establishes the left-zero alternative. In the exceptional case, the old invariant at 156 says that the destination scans one.

These facts prove I inductive in one step. They concern arbitrary tape contents; no blank-tape hypothesis or sampled execution is hidden in them.

3.3 One-step and three-step blocks

Outside logical controls 52 and 176, the new row is exactly the old row with its successor renamed by ϕ. These transitions take one target step. At either exceptional logical control, a scanned one has the same continuation, $1R150$, so again one target step suffices.

When the old control is 52 scanning zero, the invariant says that the preceding cell is one. The target follows

$$52 \xrightarrow{0L} 176 \xrightarrow{1R} 129 \xrightarrow{0R} 154.$$

When the old control is 176 scanning zero, its image starts at 52 and the invariant says that the preceding cell is zero. The target follows

$$52 \xrightarrow{0L} 176 \xrightarrow{0R} 185 \xrightarrow{0R} 168.$$

Each transition writes the bit it read. The complete tape is preserved, the head ends at $h + 1$, and the final control equals the mapped old successor. Intermediate configurations are not terminal. These are exactly the positive blocks required by Theorem 1.

For an old prefix of n steps, let $z(n)$ count the transitions at 52/0 or 176/0. The matching target prefix has length

$$t(n) = n + 2z(n), \qquad n \le t(n) \le 3n.$$

The supplied theorem does not need this uniform bound, but it makes the progress and runtime overhead transparent. The all-tape result follows immediately from the initial invariant and $\phi(0) = 0$.

3.4 The exact Lean theorem is checked, not only proposed

The final development under `cm215/lean/` has four conceptual layers:

1. the generic positive-stuttering theorem;
2. `MarkerTransport`, proving the invariant and local simulation from a finite row certificate;
3. `FiniteMarkerCertificate`, proving that a Boolean table scan implies that certificate;
4. the two literal tables, nineteen small check modules, and the final exact-table endpoint.

The scan checks all 303 represented labels on both symbols, including undefined rows. The actual working-state set has size 215; both symbols have a row at every working state. The finite type is large only because native labels reach 302. Of its 88 undefined labels, 87 are never targets and 254 is the unique halt target. An independent parser compared all 860 displayed Lean row branches with the two supplied native files.

The final instantiated endpoints are:

```
CM215Certificate.all_tape_halts_iff
CM215Certificate.blank_halts_iff
CM215Certificate.parent_working_states
CM215Certificate.marker_working_states
CM215Certificate.exactly_six_changed_rows
```

Lean 4.32.0 successfully compiled all 24 modules. The all-tape and blank endpoints depend only on `propext` and `Quot.sound`; there is no `sorryAx`, admitted theorem, or `native_decide`. Finite obligations use ordinary kernel reduction, with small shards to control resource use. This is a complete proof for the literal tables in the supplied standalone semantics.

The recorded serial build used a maximum child resident set of 612,680 KiB. This is one observed build of the standalone bridge, not a comparison with the full repository and not evidence that the data pilots improve performance. The build receipt binds each source file and compiler log by hash. The last facade-only rebuild changed a declaration from `def` to `theorem`; its accepted hash and clean log are recorded.

3.5 An independent finite-window check

The Python checker uses another presentation of the same local argument. It checks all $215 \cdot 2^5 = 6,880$ radius-two windows, of which 6,840 satisfy I. All 6,840 preservation and simulation checks pass: 6,824 need one step and 16 need three. At controls 52, 156, and 176, respectively, 16, 16, and 8 windows are excluded by the invariant.

A radius-two window suffices because a one-step invariant postcondition looks at most one cell from the new head, and the new head moved at most one cell. The three-step blocks touch only the current and preceding cells. Outside the represented window both executions leave the tape unchanged. Thus this check covers universal local premises rather than finitely many samples of a long run. Its receipt explicitly remains marked as an ordinary Python check; the separate Lean receipt is the formal evidence.

The checker also verifies the CM215 dense/native bijection, the unique halt, and the six changes. The direction counts are $(L, R) = (82, 348)$ for K215 and $(83, 347)$ for CM215. These counts show that CM215 is not merely a state-renaming of K215, even allowing a global left/right reflection.

3.6 The remaining repository adapter

The native theorem uses whole tapes $\mathbb{Z} \to \{0, 1\}$ and halt label 254. The existing formal development uses `KZip.Conf` and the K230-label sparse table, then relates that table to the dense `Machine 215` [13, 14, 15, 2]. On the native working/terminal labels, the relevant renaming is

$$301 \mapsto 102, \qquad 302 \mapsto 120, \qquad 254 \mapsto 230,$$

with all other used labels fixed. This renaming is injective on the actual working states plus the halt, not on every slot of `Fin 303`; unused slots must not be mistaken for extra machine states.

A clean integration should:

1. interpret the finite-support normalized zipper as a whole tape, fixing a relative head origin;
2. prove one-step and terminal correspondence on valid working/halt labels;
3. lift that correspondence to finite runs and blank halting;
4. reuse the repository's existing dense renaming bridge;
5. compose equation (1) with `KEndpoint215.blank_halts_iff_zfc_refutation`, then negate the equivalence.

Full tape equality in the marker proof makes the adapter independent of the source language or register interpretation. For blank halting, finite-support tapes suffice; one does not need to embed every arbitrary infinite tape into the finite-list zipper.

This adapter and the final composed repository endpoint are not supplied as completed formal proofs. The supplied exact native theorem is the substantial local bridge; its stronger all-tape statement is already checked. Keeping this distinction avoids relabelling a standalone theorem as a newly compiled repository ZFC endpoint.

4 Two immediate data simplifications with exact equality audits

These pilots shorten generated source without changing the machine, numerical calculus, checker contract, or existing certificate entries. They are appropriate first changes because their mathematical meaning can be isolated by definitional equality. Their effect on total proof-checking time is an empirical question left to the upstream build.

4.1 Lean: hash-cons the two literal trees

The current `KSource215Data.lean` contains expanded constructor literals for `t215` and `tD`, each with 4,751 primitive leaves [11]. Many constructor subtrees repeat. The pilot introduces transparent named definitions for shared nodes and uses their names at the two roots. It finds 202 unique nodes for `t215`, 283 for `tD`, and 427 in their union; 58 are shared between the roots.

Measure	Value
Original complete module	456,548 bytes
Shared-tree complete module	239,744 bytes
Saving	216,804 bytes (47.49%)
Existing forward certificate entries	2,240, unchanged
Existing forward checking shards	16, unchanged

The generator checks complete structural reconstruction of both trees and restores the replaced definitions to verify that all other source bytes are unchanged. Existing theorem statements, certificate entries, and `rfl` root identities are retained. The shared representation is ordinary definitional abbreviation; it does not add an interpretation function whose semantics must be proved afresh.

The independent audit uses a freely generated constructor datatype, keeps both original literal trees, and checks

```
theorem exact_t215 : compact_t215 = original_t215 := by rfl
theorem exact_tD : compact_tD = original_tD := by rfl
```

Lean 4.32.0 checked both theorems with no axioms. The source, pilot, generator, audit, and receipt are under `proof_simplification/`. This establishes exact tree-data equality. It does not substitute for compiling the changed original module against its imports and recompiling its consumers.

The expected benefit is a smaller and easier-to-regenerate data module. Kernel unfolding may improve or worsen depending on sharing and elaboration behavior. Before adopting it, measure the existing serial build's wall time and peak memory, retain the source hashes, and rebuild the affected closure without reusing objects whose sources changed.

4.2 Rocq: encode Boolean paths as positive binary numerals

The current `Cert215Offsets.v` is dominated by repeated explicit Boolean lists [12]. Keep the certificate's exported type and encode a path $(b_0, \ldots, b_{k-1})$ as

$$\operatorname{code}(b_0, \ldots, b_{k-1}) = 2^k + \sum_{i=0}^{k-1} b_i 2^i.$$

The most significant one is a length sentinel, while the least significant bit is the first list element. The decoder is transparent:

```
Fixpoint certPath (p : positive) : list bool :=
  match p with
  | xH => []
  | xO q => false :: certPath q
  | xI q => true :: certPath q
  end.
```

Thus the empty path is 1, [0] is 2, [1] is 3, [0, 1] is 6, and [1, 0] is 5. The sentinel preserves length and zero bits at either end; it avoids the ambiguity of a bare integer bit pattern.

Measure	Value
Original certificate module	1,499,170 bytes
Compact certificate module	229,933 bytes
Saving	1,269,237 bytes (84.66%)
Entries, offsets, successor indices, and order	All 2,240 preserved

The exported object remains a

```
list (list bool * list bool * Z * N * N)
```

so the existing offset-aware checker and its soundness theorem need no new certificate language. The audit contains the entire original literal certificate and the compact definition, then proves:

```
Theorem certificate_exact : Cert215 = Cert215Literal.
Proof. reflexivity. Qed.
```

Coq 8.18.0 compiled the audit with a closed global context; independent `coqchk` returned zero. To avoid relying on two outputs of the same possibly mistaken generator, the reconstructed original was also hash-compared with a separately retrieved copy of the pinned upstream file. That binding is recorded in `proof_simplification/rocq_pilot/upstream_binding.json`.

The large equality audit is retained as evidence, not proposed as part of every routine upstream build. The adopted data module is the 229,933-byte compact file. Logical checking still has 2,240 entries. The exact artifact is verified, but a full replay with the repository's Rocq 9.1.0 toolchain remains the adoption gate. No speedup is claimed from the Coq 8.18.0 experiment.

5 A deeper refactor: one resolved control-flow semantics

5.1 Account for simplifications already adopted

The current repository already reports a 185-module primary Lean closure, reduced from 204, and contains shared semantic work that should not be proposed as new [4]. The guarded implication schemas and deduction abstraction shrink the rho8 development to 35,185 bytes. The shared enumerator theorem handles valid states between candidates, while selector and AX-word helpers reduce duplication. The site-coverage facade composes 105 checked blocks directly and is 14,916 bytes; its nine batch files and 4,751 leaf checks are unchanged. Earlier phases supply forward-only source-certificate symmetry and associated check shards.

The maintained `KEndpoint215` therefore has the right high-level shape already: compose the dense correspondence, runtime box lemmas, site coverage, boot run, boot relation, and source calibration.

Wrapping that composition in another lemma would make its display shorter without shrinking the mathematical burden. The next valuable semantic abstraction sits below it.

5.2 Proposed graph interface

For source changes that preserve the resolved primitive control-flow graph, prove source-tree equivalence once through that graph. Number primitive occurrences by ordinal and record, for each occurrence:

1. its register operation and register index;
2. its normal successor and, for a decrement, its zero/nonzero successors;
3. the destinations obtained after resolving BREAK, CONTINUE, and other structural control transfers;
4. terminal behavior and any remaining control-only transitions.

Then prove a generic correspondence between source configurations and graph configurations for arbitrary register values. Two differently associated trees whose resolved graphs agree can share their semantic proof by the identity occurrence map.

This is stronger than equality of primitive leaf words. Reassociating loops can change a BREAK target while leaving the same leaf sequence. Such programs can have different behavior. The certificate must compare the resolved guarded edges and terminal destinations, not only instructions. Similarly, silently chasing structural control transitions requires either a termination proof for that chase or explicit representation of silent divergence. A silent cycle must not be discarded as if it reached the next primitive.

This interface differs from the current 2,240-entry source-to-`DagPerm` offset witness. For two trees with the same resolved graph, no paired-address search, segment offset relation, or new source-to-calculus proof is necessary. Tree shape remains an implementation concern; the primitive graph carries the semantics.

5.3 What this refactor would and would not remove

It is particularly useful for rebalanced and reencoded sources and for generating a family of source representations with one shared logical proof. It can also express carefully guarded dead-code splices, provided their reachability assumptions are proved.

It does not by itself certify the binary machine. The current site batches still check occurrence-specific symbolic runs through expressions of the form

```
allSites (KSim.checkEdge tm215) ... = true
```

for all 4,751 primitive occurrences. Removing those checks requires a separate compiler-correctness theorem or a machine-level transport certificate. The existing facade refactor makes their composition shorter, not their content disappear. Similarly, the 95,714-step blank prefix remains a checked machine-specific fact unless a new machine inherits it through an equihalting bridge or receives a proved bootstrap contract.

A useful architecture therefore has two reusable semantic interfaces: source trees to resolved primitive graphs, and physical machine transformations to positive-stuttering certificates. The first handles source representation; the second handles nearby implementations. Neither needs to replace the existing consistency calibration.

6 Comparison of the main 215-state alternatives

The question "easier to prove" depends on whether one reuses K215. The recommendation here is based on additional obligations relative to the currently proved endpoint. CM215 has the smallest supplied bridge. Rebalanced R41 is the next useful target for a source-graph abstraction. The other alternatives change an interface that currently carries substantial semantic work [6, 7, 8, 9, 10].

Candidate	Relevant change	Additional proof burden
CM215	Six binary rows; unchanged source and seed	Three-clause invariant and 1/3-step transport. Exact standalone Lean bridge supplied here.
Rebalanced R41	Nineteen rows; depth 47 to 41; 4,120 addresses	Resolved graph equivalence and new physical/decoder correspondence. Same 95,714-step seed helps.
commands120	Command permutation	Command/compiler and source certificates, physical sites, and changed startup. Permutation alone is not a shorter proof.
composed	New allocation of the same source	Source semantics can be reused, but a physical correspondence is still needed. No equally small local bridge supplied.
reencoded	Seventeen rows; 4,519 addresses	Graph/encoding bridge and changed seed with a nonzero auxiliary counter. Startup is 716,307 steps.
zero-spare Z215	Changed start and one write bit	Append-zero engine, two truth heads over a protected tail, and a new seed/scheduler theorem.
Direct-transfer ST215	Smaller transfer command and new startup	Transfer compiler and partial AX-prefix calibration, protected-tail credit, physical proof.

Some numerical comparisons are useful but should not be treated as proof-complexity measures. K215 has 4,751 primitive occurrences and 6,176 guarded runtime cases. Rebalanced R41 retains those inventories while reducing source depth from 47 to 41. The inspected physical derivations have 1,698,790 symbolic rules for the primary and 1,640,317 for R41. That reduction can help checking costs, but it does not erase the semantic layers.

CM215's inspected physical derivation has 1,701,542 rules, slightly more than the primary. This is not a reason to reject it as the easiest additional theorem: the local bridge avoids replaying that derivation. Its correctness does not need a new 95,714-step startup proof, since the all-tape equivalence begins at the designated start control.

Z215's native start is 74 and its changed row is 198/0, which writes zero. Its 117,004-step startup reaches the logical seed $[1, 0, 5, 0, 0, 0, 0, 0, 0]$. The protected tail under two truth heads is a semantic debt that must be carried through the scheduler. A one-bit table change is therefore not automatically a one-line correctness proof.

Direct-transfer reduces the source to 4,732 occurrences and 6,151 guarded cases. Its 70,506-step startup matches only a partial AX prefix, followed by 177 source transitions and a ten-head protected credit argument. Reencoded reaches $[4, 0, 0, 0, 1, 0, 0, 0, 0]$ rather than the primary seed. These alternatives are plausible design directions, but their new seeds or compiler commands prevent direct reuse of the complete baseline proof without additional theorems.

These candidate statistics are from the pinned repository packets, not fresh rebuilds of every alternative. This investigation independently checked the exact CM215 local correspondence and supplied its kernel proof; it did not compile new full endpoints for the other candidates.

7 The search for 214 states: completed negative results

No valid 214-state table is supplied. The experiments below narrow specific search families and identify a more promising next transformation. They neither prove minimality nor rule out a 214-state machine outside their bounds.

7.1 Finite-window local rewrites on CM215

The supplied search independently implements finite-window invariant and joining-diagram ideas already used in repository research [16]. The completed CM215 instance computes an inductive radius-seven abstraction from all initial tapes at control zero. It contains 972,399 admissible control/window pairs. Proposed rewrites must reach a common endpoint with identical head and tape, with at most 97 steps on each side. Unknown cells beyond the justified window are not assumed to contain zero.

Completed search quantity	Result
Nonidentity single-row alternatives	371,090
Accepted local stuttering rewrites	483
Two-state folds admitted by the rewrite closure	0
Noninitial controls considered for deletion	214
Entrance-redirection actions tried, with early pruning	186,620
Complete state deletions found	0

The first count is $430(4 \cdot 216 - 1)$: two possible writes, two directions, and 216 destinations including the halt, excluding the unchanged action. The deletion search changes external entrances independently, requiring a join at a retained old control before executing another changed entrance. It stops considering a proposed deletion when an entrance has no valid option. Thus 186,620 is an action count under first-failure pruning, not an exhaustive count of all simultaneous multi-row patches.

A four-state positive-control machine has a known merge/deletion. The same code finds it. This checks that the success conditions are not accidentally empty; it does not turn the search implementation into a verified optimizer.

Three deletions have partial local progress. For dropping 31, entrance 13/1 can redirect as $1L81$, $1L107$, or $1L128$, but 20/1 fails the bounded condition. For dropping 40, $27/1 \mapsto 0L302$ passes but 54/0 fails. For dropping 52, $156/1 \mapsto 1R155$ passes but 165/0 fails. None is a candidate machine: all entrances and all reused helper behaviors must be justified together.

The final run took about 2.47 seconds and 311,720 KiB peak resident memory in the recorded environment. The code, inputs, positive control, resource record, and detailed accepted/rejected-family receipt are under `reductions/`. Failure within this family can result from too weak an invariant, a longer necessary join, or the need for a coordinated rewrite that never rejoins the unchanged machine at an allowed checkpoint.

7.2 Disjunctive source abstraction and dead-code splicing

A second experiment tracks each of nine registers in the classes 0, 1, and ≥ 2, retaining a full set of abstract stores rather than taking only a product join at each location. There are $3^9 = 19,683$ stores. From the calibrated seed $(4, 0, \ldots, 0)$, the fixed point contains 5,455,668 primitive-location/store pairs.

It confirms unreachable ordinals 4672, 4673, and 4734, and always-zero decrement tests at 4671 and 4733. These facts were already found by the repository's earlier product-domain analysis; the stronger

relational experiment finds no new dead source region. Its exact splice check preserves the labels and 6,168 guarded edges of the retained graph after skipping the two zero decrements.

Source variant	Primitive occurrences	Distinct constructor subtrees
Original	4,751	202
First loop removed	4,748	203
Second loop removed	4,749	204
Both removed	4,746	205

Deleting five primitive occurrences increases the shared constructor DAG from 202 to 205 nodes. This is a concrete reason not to optimize leaf count in isolation: physical encoding benefits from repeated subtrees, and a smaller unfolded program can require more distinct encoded components. The experiment supplies a source-level, seed-reachable abstraction result and guarded splice check. It does not supply a binary allocation or proof-assistant theorem for the modified source.

7.3 A concrete next target: reorder the selector tests

The existing CM215 selector spends control 176 to distinguish the old roles 52 and 176. Its two downstream controls have a shared zero branch:

Control	Read zero	Read one
154	$0R166$	$1R12$
168	$0R166$	$1R98$

The predecessor marker matters only when the *next decoded bit* is one. A coordinated rewrite could first inspect that next bit, immediately take the common zero continuation, and inspect the predecessor marker only on the one branch. This is the most concrete next allocation target identified here.

More precisely, start at an old logical control 52 or 176 scanning zero at position h. Let $m = \tau(h-1)$ and $b = \tau(h+1)$. The two old steps preserve the whole tape and finish at $h+2$ with control

condition	required control
$b = 0$	166
$b = 1,\ m = 0$	98
$b = 1,\ m = 1$	12.

The invariant relates m to the old logical role. A new implementation of this three-way contract must also preserve control 52's current-one behavior, every other use of borrowed helpers, and blank-start behavior. Reordering reads can require extra head-travel controls; the shared zero continuation alone does not guarantee a state saving.

The current one-source-step map does not automatically prove this fused two-step transformation. A refined configuration map or a common-checkpoint machine with positive blocks is needed. The generic transport theorem remains useful after defining the checkpoint semantics. A plausible optimizer should search allocation and proof together: emit a complete table, a state census, the invariant, and all local block certificates, then independently replay them before claiming 214.

The literal selector row pair $0R185/1R129$ has no second exact copy in CM215. Therefore merely redirecting to an identical existing selector is not an available one-state saving. The new target is a coordinated decision-order transformation, not an omitted trivial merge.

8 Recommended incorporation sequence

1. **Import the reusable transport theorem.** Keep the small standard-library Lean and Rocq versions independent of the large ZFC development. Their client contract should remain explicit about progress and terminal preservation.
2. **Integrate the CM215 native bridge.** Add the state/tape/run adapter, compose with the established K215 endpoint, and publish typed positive and negated endpoints. Retain the exact native/dense table hashes, including the already verified six-row difference.
3. **Adopt data compaction in separate changes.** First share the Lean trees, then compact the Rocq paths. Rebuild the respective affected closures, rerun endpoint clients and assumption reports, and compare time and memory against the same pinned baseline. Keep the standalone equality audits as reproducible evidence.
4. **Prove the resolved-CFG interface.** Exercise it on rebalanced R41, where the seed and primitive inventory remain unchanged. Preserve all guarded and control-only edges, not only leaf order.
5. **Continue the 214 search with coordinated rewrites.** Use the delayed-marker contract as a concrete target. Optimize allocated binary states and certificate complexity jointly; do not infer a state reduction from source leaf counts.

This ordering yields useful checked improvements before undertaking a new compiler or seed theorem. It also leaves each adoption independently reviewable: a generic theorem, a local physical bridge, two exact-data changes, and a subsequent semantic refactor.

9 Reproduction and artifact inventory

The archive contains source, exact input tables, generators, receipts, compiler logs, independent comparison records, the report source, and this PDF. Compiled proof objects and development toolchains are omitted. Paths below are relative to the archive root.

Directory / file	Contents
`cm215/`	Exact table inputs, independent Python checker, label audit, reproducible generation/build scripts, 24-module Lean closure and accepted logs.
`proof_simplification/`	Generic Lean/Coq transport, original and shared Lean data, compact Rocq data, both full equality audits, generators and receipts.
`reductions/`	Completed local search, positive control, disjunctive source analysis, guarded splice, independent proof/table review.
`reference/`	Pinned maintained endpoint, table, zipper, dense bridge, and current simplification summary.
`README.md`	Quick-start commands, evidence map, limitations, and adoption instructions.
`PROVENANCE.json`	Repository pin, toolchain versions, status and input hashes.
`SHA256SUMS`	Integrity manifest for all other packet files.

With Python 3 and Lean 4.32.0 installed, run the independent check and the standalone kernel build:

```
python3 cm215/check_cm215_local.py
python3 cm215/check_repository_labels.py
python3 cm215/build_lean.py --lean /absolute/path/to/lean
```

The build is serial and memory-capped by default. It imports only Lean's bundled libraries and requires no Mathlib checkout. To reproduce the generated literal tables and finite checks, use `cm215/generate_certificate.py`; it verifies the input hashes before generation. Generation alone is not kernel checking.

For the generic Coq theorem and compact-path audit, use installed Coq 8.18.0 and `coqchk`, passing absolute executable paths to the supplied scripts. The accepted receipts identify this toolchain, not Rocq 9.1.0. The original repository endpoints have not been rebuilt in this packet. For search reproduction, the Node.js commands and exact radius/length bounds are in `reductions/REDUCTION_NOTES.md`.

The article and packet status take precedence over any historical notes inside unchanged upstream attribution files. Historical receipts are preserved as records of their actual run, while the manifest binds the delivered source bytes. No rejected or incomplete 214-state table is labelled as a candidate.

References

[1] ProveIt, primary K215 packet, pinned snapshot. candidate-k215/README.md.

[2] ProveIt, maintained Lean endpoint. KEndpoint215.lean.

[3] ProveIt, formal verification overview. k225-formal/README.md.

[4] ProveIt, current proof simplification phase. Shared lemmas and one enumerator theorem.

[5] ProveIt, conditional-marker alternative and original proof. CM215 README; marker215-proof.tex.

[6] ProveIt, comparison of alternatives. alternatives/README.md.

[7] ProveIt, rebalanced candidate. rebalanced/README.md.

[8] ProveIt, reencoded candidate. reencoded/README.md.

[9] ProveIt, zero-spare candidate. zero-spare/README.md.

[10] ProveIt, direct-transfer candidate. direct-transfer/README.md.

[11] ProveIt, maintained Lean source data. KSource215Data.lean.

[12] ProveIt, Rocq offset certificate. Cert215Offsets.v.

[13] ProveIt, normalized zipper semantics. KZip.lean.

[14] ProveIt, K230-label 215-state table. KTable215.lean.

[15] ProveIt, dense-machine representation bridge. KDenseBridge.lean.

[16] ProveIt, earlier finite-window, folding, and source-abstraction experiments. abstract_constants.py; abstract-cap4/result.json; rewrites-primary/result.json.